

需求描述以及其明确界定,是软件系统构建过程中非常重要的起点。在这一阶段,需求分析工作会产出一些关键成果,其中主要包括领域模型以及用例模型。这些成果能够帮助开发人员更好地理解系统结构,并且为后续设计工作提供基础。其中,领域模型也被称为概念模型或类模型,本质上是一种类图形式。不过与常规类图不同的是,该模型只包含类的属性以及职责说明,而不涉及具体的方法定义。

当我们获得用户以自然语言形式表达的需求之后,还需要对这些需求进行整理与分析。原始需求往往没有经过系统化组织,其中可能存在表达模糊、信息零散甚至不一致的情况。有些需求描述还可能难以实现,或者缺乏必要的细节说明。在这种情况下,如果直接依据这些原始需求开展系统设计,就容易出现理解偏差。因此,设计人员需要在充分理解现有需求内容的基础上,对需求进行进一步整理与归纳。通过这种方式,可以将原本较为混乱的需求信息转化为结构清晰的需求描述,从而为后续系统设计与建模工作打下更可靠的基础。

在完成需求分析阶段后,我们应该能得到如下软件制品^[22]:

- (1) 用例图 (制品编号: RA-1) ;
- (2) 高层文本用例 (制品编号: RA-2) ;
- (3) 详细文本用例 (制品编号: RA-3) ;
- (4) 概念类模型 (制品编号: RA-4) ;
- (5) 用例序列图 (制品编号: RA-5) ;
- (6) 用例系统操作的契约 (制品编号: RA-6) 。

在 2.2.1 至 2.2.6 小节中,本文详细阐述 RA-1 至 RA-6 制品的规范,并展示一个典型的医疗领域:门诊挂号场景的制品内容。需求分析阶段产出制品的思维导图如图 2.1。

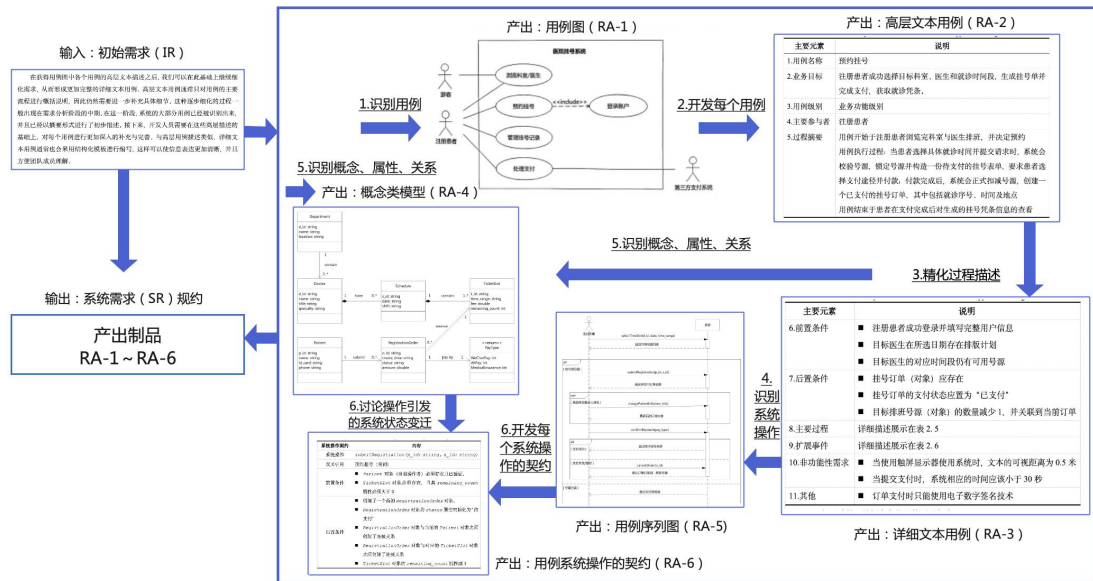


图 2.1 需求分析阶段产出制品的思维导图

Fig2. 1 Mind map of the output products in the requirements analysis stage

2.2.1 用例图开发（RA-1）

用例是特定参与者为达成其具体的业务目标，通过与系统交互所执行的一系列完整的活动序列；该活动序列包括从业务开始、发展到结束期间的所有活动，并获得有业务价值的、稳定持久的结果；同时包含了成功和失败的情形。在实践过程中，尽可能多地从参与者与系统之间的交互活动中，发现可作为候选的用例，根据用例的定义，从候选用例的集合中确定合适的用例。用例的识别需要满足其对下列要素的刻画：(1)完整的动作序列；(2)具体的业务目标；(3)成功与失败的场景^[22]。

在完成用例的发现与识别之后，我们可以得到一组系统用例集合。在此基础上，开发人员便能够进一步绘制用例图，从而对系统功能进行整体展示。用例图主要通过描述参与者、用例以及它们之间的关系来体现系统功能结构。这些关系主要包括三种类型，其一是参与者与参与者之间的关系，其二是用例与用例之间的关系，其三是参与者与用例之间的交互关系。通过这些关系的组合，可以较为直观地反映系统在实际使用过程中的功能交互方式。在 UML 建模中，用例图通常包含四类基本元素，分别是系统边界、参与者、用例以及关系。系统边界用于表示系统范围，参与者表示与系统进行交互的外部角色，用例则用来描述系统能够提供的具体功能，而关系用于说明这些元素之间的联系。

用例图中主要刻画三类关系：参与者与用例之间的关联关系：关联关系，用直线表示，参与者之间的关系：泛化关系，用直线+三角箭头表示，箭头指向父类型，用例与用例之间的关系：包含或扩展关系，用虚线+箭头+<<include>>或<<extend>>标注表示。各关系的 UML 图示如图 2.2 所示。

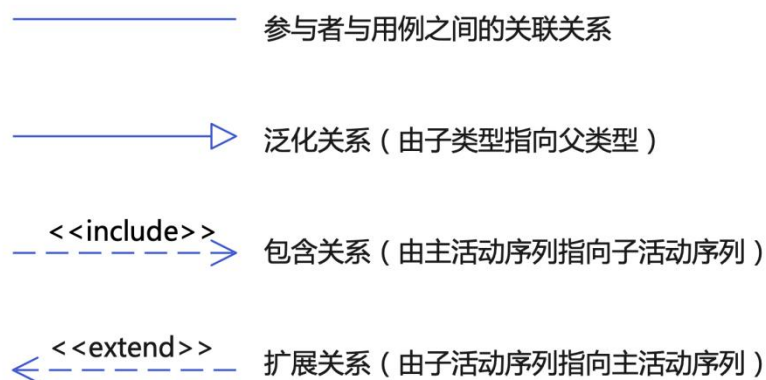


图 2.2 用例图中常用的关系表达

Fig2. 2 Common relationship expressions in use case diagrams

通过构建用例图，软件开发团队能够更清楚地了解系统需要实现的功能范围，并且对系统整体功能结构形成较为直观的认识。这对于后续系统设计以及功能实现具有重要的参考价值。图 2.3 展示了门诊挂号场景的用例图示例。

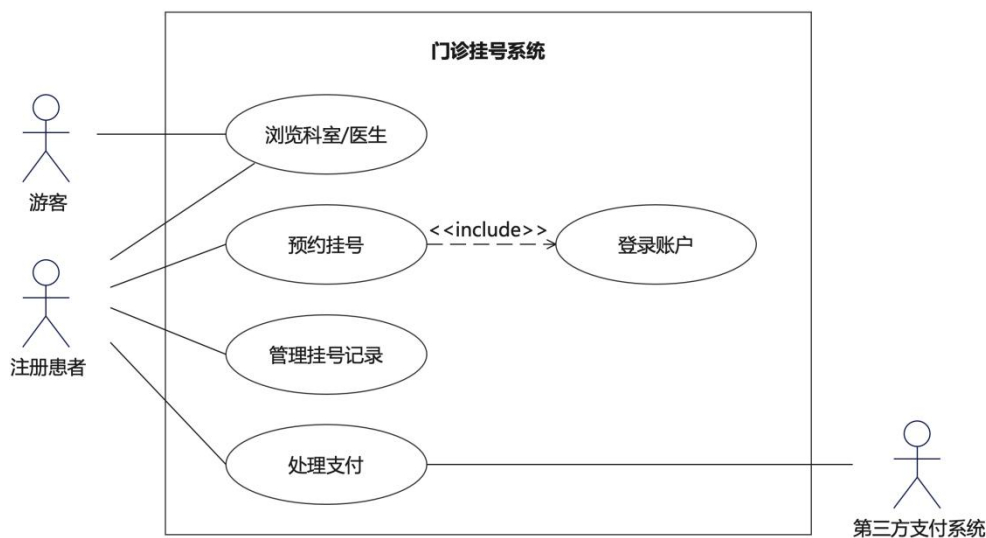


图 2.3 门诊挂号系统用例图示例

Fig2. 3 Example of a use case diagram for the outpatient registration system

2.2.2 高层文本用例开发（RA-2）

通过用例图，我们能够了解系统包含哪些主要功能，并且可以看到不同参与者与系统之间的交互关系。不过，用例图主要用于展示整体功能结构，却无法体现用例内部的具体动作流程。而这些动作序列往往包含了业务处理的关键步骤，是理解业务逻辑的重要信息。为了完整刻画系统行为，我们需要结合用例的文本描述，将每个用例内部的操作和事件进行详细分析。这些文本描述通常采用结构化模板，将主要流程、扩展流程以及前置条件和后置条件明确列出。

因此，与用例图相比，每个用例对应的文本描述（RA-2 与 RA-3）能够提供更加丰富的需求细节。通过这些描述，开发人员可以更清楚地看到用例在实际执行过程中需要完成哪些操作，并且能够理解业务流程的具体内容。

在实际需求建模过程中，用例文本通常采用结构化方式进行编写。这样的结构化描述能够让信息更加清晰，并且有助于对用例内容进行组织与展示。通过这种方式，开发团队能够更容易理解用例的业务过程，并且为后续系统设计提供更加完整的需求依据。一种最广泛使用的用例文本结构模版^[23]如下：

Use Case: <number><the name should be the goal as a short active verb phrase>
Goal in Context: <a longer statement of the goal, if needed>
Scope: <what system is being considered black-box under design>
Level: <one of: Summary, Primary task, Subfunction>
Primary Actor: <a role name for the primary actor, or description>
Priority: <how critical to your system / organization>
Frequency: <how often it is expected to happen>

基于此，本论文采用的高层文本用例描述模版如表 2.1 所示^[22]。

表 2.1 用例高层文本描述模版

Table2. 1 The template of overall textual use case

主要元素	说明
1.用例名称	用例名称与先前识别的用例名称一致
2.业务目标	主要参与者执行本用例期望达到的业务目标
3.用例级别	业务功能级别或子业务级别。在高层文本描述中，一般只涉及业务功能级别的用例，在进行细化后才会抽取子业务级别的用例
4.主要参与者	主动触发用例、驱动用例执行完整活动序列并实现该参与者期望的业务目标的参与者

续表 2.2 用例高层文本描述模版

Table2. 2(Cont.) The template of overall textual use case

主要元素	说明
5.过程摘要	此处简要描述用例成功执行的基本过程，通常用于快速了解当前用例的主题和范围。根据识别用例的方法，此处需要表达出用例作为一个“完整的动作序列”应具有的特征，即需要描述用例开始和结束的行为，以及发生在始末之间的过程。建议显式地使用“开始于”“结束于”“执行过程”等关键字来辅助描述用例执行时的过程摘要

如表 2.2 所示，以门诊挂号场景中“预约挂号”的用例为例，可将其高层文本描述开发如下。

表 2.3 用例“预约挂号”的高层文本描述

Table2. 3 The description of overall textual use case "Appointment Registration"

主要元素	说明
1.用例名称	预约挂号
2.业务目标	注册患者成功选择目标科室、医生和就诊时间段，生成挂号单并完成支付，获取就诊凭条。
3.用例级别	业务功能级别
4.主要参与者	注册患者
5.过程摘要	用例开始于注册患者浏览完科室与医生排班，并决定预约 用例执行过程：当患者选择具体就诊时间并提交请求时，系统会校验号源，锁定号源并构造一份待支付的挂号表单，要求患者选择支付途径并付款；付款完成后，系统会正式扣减号源，创建一个已支付的挂号订单，其中包括就诊序号、时间及地点 用例结束于患者在支付完成后对生成的挂号凭条信息的查看

高层文本描述通常应用在需求分析的早期阶段。在这一阶段，开发人员已经完成了用例的初步识别，并且得到了系统的用例集合。通过高层文本描述，可以对各个用例的主要执行步骤进行整体性的说明。虽然这种描述方式不会涉及过多细节，但能帮助开发人员快速了解每个用例的大致业务流程。团队成员在需求分析初期就能够对系统功能形成基本认识，并且为后续更详细的需求描述打下基础。

2.2.4 详细文本用例 (RA-3)

在获得用例图中各个用例的高层文本描述之后，我们可以在此基础上继续细化需求，从而形成更加完整的详细文本用例。高层文本用例通常只对用例的主要流程进行概括说明，因此仍然需要进一步补充具体细节。在这一阶段，系统的大部分用例已经被识别出来，并且已经以摘要形式进行了初步描述。接下来，开发人员需要在这些高层描述的基础上，对每个用例进行更加深入的补充与完善。与高层用例描述类似，详细文本用例通常也会采用结构化模板进行编写，这样可以使信息表达更加清晰，并且方便团队成员理解。

在开发详细文本用例时，最关键的一项工作是对高层描述中的“过程摘要”进行扩展。通过对这一部分内容进行补充，可以进一步明确用例执行过程中的关键要素，包括前置条件、后置条件（成功保证）、主要执行流程以及扩展事件等。这些内容共同构成了详细文本用例的核心结构，也为后续系统设计与实现提供了更加完整的需求依据。我们可以采用如表 2.3 所示的结构化描述模版^[22]，其中主要元素 1 至 5 项同表 2.1。

表 2.4 用例详细文本描述模版
Table2. 4 The template of detailed textual use case

主要元素	说明
6.前置条件	交代用例在开始执行之间必须满足的条件
7.后置条件	描述用例在执行完毕之后需要得到的成功保证
8.主要过程	本部分是对“5.过程摘要”的细化，主要描述典型的、理想状态下用例成功执行的场景。其表述过程主要考虑“主要参与者”与“系统”两个对象之间的交互，即考虑主要参与者的请求序列及系统对相关请求的响应
9.扩展事件	本部分是对“8.主要过程”的补充，主要描述用例执行过程中异常事件发生时的处理过程，或主要场景中针对某些活动序列的可替换（分支）序列，其描述包活三个部分：编号、触发条件、处理过程
10.非功能性需求	描述用户对响应时间、系统接口、服务质量等方面的需求
11.其他	可描述用例执行频率等能够辅助设计决策的信息

如表 2.4 所示，根据上述模版，继续开发“预约挂号”用例的详细文本用例，其中主要元素 1 至 5 项同表 2.2。

表 2.5 用例“预约挂号”的详细文本描述

Table2. 5 The description of detailed textual use case "Appointment Registration"

主要元素	说明
6.前置条件	■ 注册患者成功登录并填写完整用户信息 ■ 目标医生在所选日期存在排班计划 ■ 目标医生的对应时间段仍有可用号源
7.后置条件	■ 挂号订单（对象）应存在 ■ 挂号订单的支付状态应置为“已支付” ■ 目标排班号源（对象）的数量减少 1，并关联到当前订单
8.主要过程	详细描述展示在表 2.5
9.扩展事件	详细描述展示在表 2.6
10.非功能性需求	■ 当使用触屏显示器使用系统时，文本的可视距离为 0.5 米 ■ 当提交支付时，系统相应的时间应该小于 30 秒
11.其他	■ 订单支付时只能使用电子数字签名技术

“8.主要过程”举例内容的具体展示如表 2.5。

表 2.6 用例“预约挂号”的“8.主要过程”

Table2. 6 "8. Main Processes" of the use case "Appointment Registration"

参与者活动	系统相应 (What/How)
0.用例开始于注册患者浏览完目标科室与医生排班，并准备进行预约	
1.注册患者从医生排班列表中选择期望的就诊日期和具体时间段，启动预约挂号结算流程	2.系统首先校验该时间段的号源可用性。若有余号，系统锁定 1 个该号源的额度，并生成当前挂号单的表单，进入填写订单环节。挂号表单将展现拟预约的科室、医生姓名、就诊时间及挂号费金额。系统会自动载入并显示该账户默认的就诊人信息

续表 2.7 用例“预约挂号”的“8.主要过程”

Table2. 7(Cont.) "8. Main Processes" of the use case "Appointment Registration"

参与者活动	系统相应 (What/How)
3. 患者确认就诊人信息无误后, 选择并确认支付方式 (如微信支付、支付宝支付等), 随后选择提交支付	4. 系统根据当前患者的支付选择转入支付流程, 并进入“处理支付”用例。 在支付成功后, 返回本用例, 并将挂号订单状态置为“已支付”。在订单完成支付后, 系统正式扣减对应的号源, 并生成包含就诊序号、就诊地点、就诊时间的电子挂号凭条。同时, 系统向患者预留的手机号发送短信告知预约成功的通知
5. 用例结束于注册患者在支付完成后, 对生成的电子挂号凭条中有关就诊序号、时间及地点等信息的查看	

“9.扩展事件”举例内容的具体展示如表 2.6, 其中, 描述项“编号”中, 前面的数字代表活动序号, 后面的字母代表扩展事件的顺序。

表 2.8 用例“预约挂号”的“9.扩展事件”描述举例

Table2. 8 Example of description of "9. Extended Event" in use case "Appointment Registration"

编号	触发条件	处理过程
1a	系统校验发现目标医生在所选时间段的号源剩余数量等于 0	1. 系统弹出提示“该时间段号源已被抢空” 2. 系统终止当前结算流程 3. 页面返回至该医生的排班列表, 供注册患者重新选择其他有余号的时间段
3a	系统默认载入的就诊人信息并非当前患者期望的实际就诊人	1. 系统将该待支付挂号单的状态修改为“已取消”或“支付失败” 2. 系统立即释放先前为该订单锁定的 1 个号源额度, 将其退回号源池 3. 系统提示患者“支付失败/超时, 号源已释放, 请重新预约”

续表 2.9 用例“预约挂号”的“9.扩展事件”描述举例

Table2. 9(Cont.) Description of "9. Extended Event" in use case "Appointment Registration"

编号	触发条件	处理过程
3b	当前患者期望使用医保账户（医保卡或医保电子凭证）进行结算	1. 患者在支付方式选项中勾选“医保支付” 2. 系统唤起医保电子凭证授权页面（或提示读取实体医保卡信息），验证该就诊人的医保身份与账户有效状态 3. 验证通过后，系统调用医院医保结算接口，根据医保统筹规则自动计算挂号费中的医保报销金额与个人自费金额，并刷新当前挂号单的金额显示 4. 系统返回执行本用例的主流程，患者继续对剩余的个人自费部分（若有）选择第三方支付并提交

2.2.4 概念类模型 (RA-4)

系统结构建模产出的产品为概念类模型。在面向对象的软件工程方法中，系统架构的静态结构一般通过类图进行表达。类图中包含了类、对象以及它们之间的关系与连接等元素，这些都是面向对象建模中的基本概念，也共同构成了概念类图的核心组成部分。概念类模型是仅包含类属性及其职责描述，不包含类方法的类图，旨在从业务场景中提取核心实体，明确这些概念的重要属性，并定义它们之间的关系。

在构建概念类模型的过程中，需要先从需求描述中提取可能存在的概念，并以此形成概念类以及其属性的候选集合。随后，开发人员还需要对这些候选元素进行进一步分析与筛选，从中识别出真正与系统业务相关的概念类。整个过程通常包括以下几个步骤：

- (1) 发现概念与属性；
- (2) 甄选概念与属性；
- (3) 开发初始概念类图。

在前面的步骤完成之后，我们已经得到了一幅初始的概念类图。不过，这一阶段的类图还只是一个尚未完全完善的模型。虽然我们已经知道系统中可能包含

哪些类，但仅仅列出这些类本身还不够。若要真正表达系统结构，还需要明确类与类之间的关系。只有当这些关系被建立起来之后，类之间才能形成完整结构。不同的结构关系，也会支持不同类型的系统功能。因此，在得到初始概念类图之后，我们还需要继续定义类之间的关系。常见的关系类型主要包括一般关系、泛化关系以及聚合或组合关系。通过这些关系，可以更清楚地表达系统中各个概念之间的联系。在前三个步骤中，我们已经构建出一幅尚未标注关系的初始概念类图。当我们进一步分析并确定概念之间存在哪些联系之后，就可以在类图中对这些关系进行标注。完成关系标注后，接下来的建模过程通常还包括以下几个步骤：

- (4) 识别关系的种类；
- (5) 明确关系的度；
- (6) 给出关系的名称和角色；
- (7) 给定关系的重数。

经过上述步骤后，我们可以开发出相关用例的概念类图。“预约挂号”用例的概念类图如图 2.4 所示。

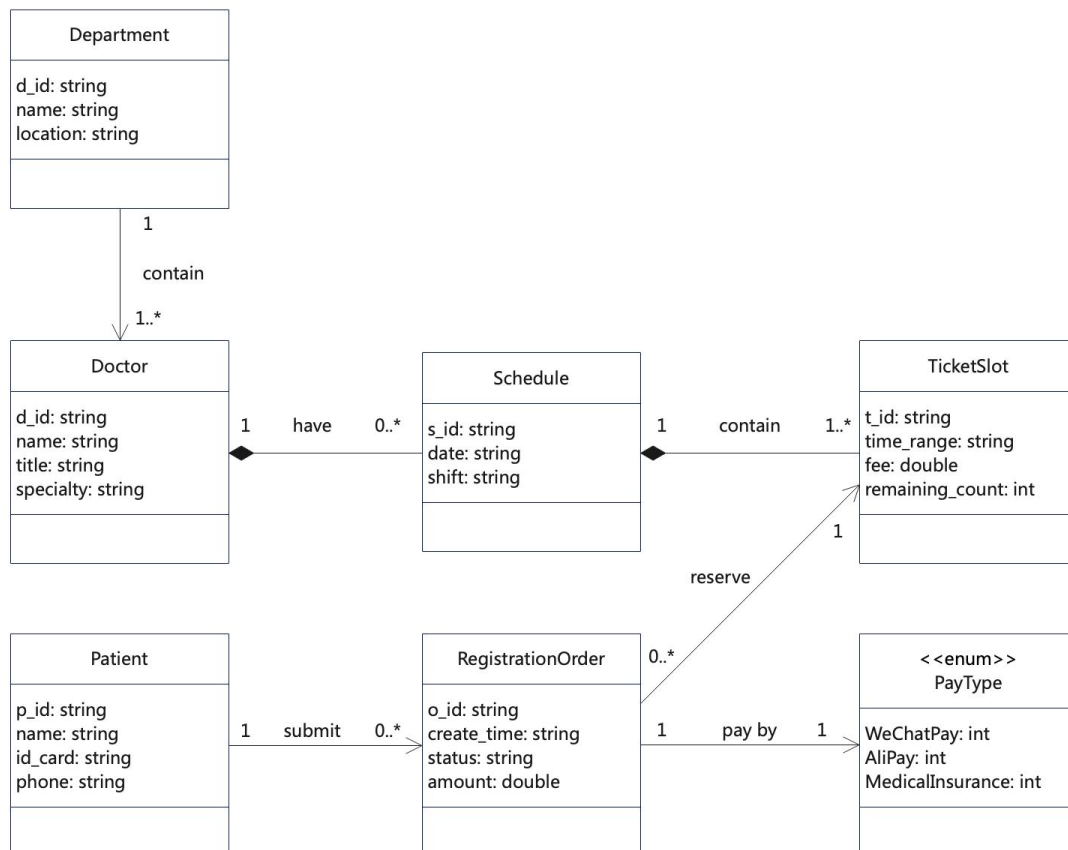


图 2.4 “预约挂号”用例的概念类图

Fig2. 4 conceptual class diagram of the "Appointment Registration" use case

2.2.5 用例序列图 (RA-5)

系统行为建模阶段最终得到的成果通常是用例序列图。用例的行为建模主要用于描述用例执行过程中参与者与系统之间的交互过程。在这一过程中，需要识别系统需要提供的具体操作，并且在这些操作的基础上，构建参与者与系统之间的交互关系图。

在本文的研究中，我们选择使用用例序列图来表达这种交互关系。用例序列图通常围绕某一个具体的用例场景展开，通过展示参与者与系统之间的信息交互顺序，来描述系统在该场景下的行为过程。与结构模型相比，这种方式能够更加清晰地体现系统在运行过程中的动态变化。在建模过程中，用例序列图的主要输入来源是用例的文本描述，尤其是其中的“主要过程”部分。这一部分通常记录了用例执行时的关键步骤，因此能够为交互建模提供重要依据。在分析这些描述时，我们主要关注三类系统事件：

- (1) 参与者发起的外部事件；
- (2) 与时间相关的事件；
- (3) 错误或异常事件。

在开发用例序列图的过程中，我们可以关注参与者、系统（或用例）、生命线、激活期、系统操作/消息、控制段等元素。用例序列图的绘制过程如下：

(1) 仔细阅读并理解详细的用例文本描述，重点关注 `system response` 部分，并且分析每一个 `response` 所对应的 `actor action`。通过这种方式，可以明确参与者操作与系统反馈之间的对应关系；

(2) 在分析 `action` 与 `response` 对应关系的基础上，对其中的系统行为进行抽象，从而识别出 `system operations`。这些操作通常可以用动宾结构进行表达，用来表示某类参与者希望系统完成某项业务功能时需要执行的具体操作；

(3) 在识别出系统操作之后，还需要为这些操作补充必要的参数。参数主要用于表示当参与者通过系统完成某项业务操作时，系统需要从参与者那里获取的数据内容；

(4) 再次结合详细文本用例中的主要流程，对整个业务处理过程进行梳理，并据此绘制用例序列图。在绘制过程中，需要明确序列图中的各类图形元素，并且特别注意 `loop` 与 `alternative` 等结构的使用。同时，应根据业务流程的实际执行

逻辑，对必要的细节进行补充，使序列图能够更加准确地反映系统行为。

“预约挂号”用例的用例序列图示例如图 2.5。

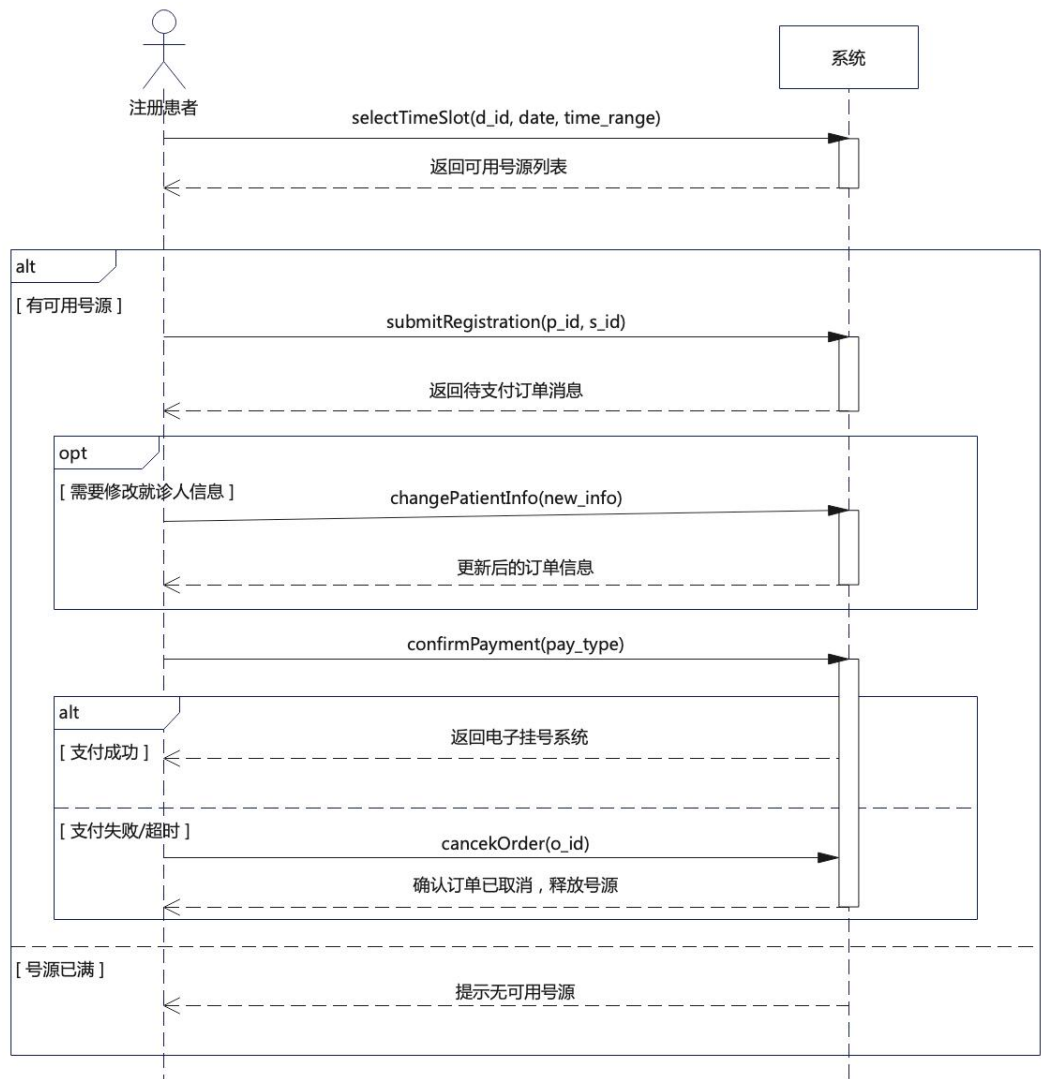


图 2.5 “预约挂号”用例的用例序列图

Fig2. 5 The use case sequence diagram of the "Appointment Registration" use case

2.2.6 用例系统操作的契约 (RA-6)

系统操作契约主要用于描述系统操作在执行前后所引起的系统状态变化。这些变化通常包括对象的创建与销毁、对象属性的更新，以及对象之间连接关系的建立或解除。通过这些变化，我们可以更清楚地理解系统在执行某个操作时内部状态是如何发生改变的。从本质上来看，系统操作正是推动系统状态发生变化的重要原因。因此，在刻画系统操作契约时，需要先理解系统状态变化的基本规律。只有明确系统在不同阶段的状态，才能更准确地描述操作执行过程中的变化情况。在具体建模时，通常会通过描述系统操作执行前后的状态来表达操作契约的核心

内容。通过这种方式，可以清晰地说明某个操作在执行之前系统处于什么状态，并且在操作完成之后系统状态发生了哪些变化，从而形成对系统行为的完整描述。

系统操作契约主要用于描述系统操作在执行前后所引起的系统状态变化。这些变化通常包括对象的创建或销毁、对象属性的更新，以及对象之间连接关系的建立或解除。通过对这些变化进行明确描述，可以更清楚地理解系统在执行某项操作时内部状态发生了哪些改变。在开发系统操作契约时，核心任务是构建霍尔三元组^[24]中的 *Pre-condition* (前置条件) 与 *Post-condition* (后置条件)。霍尔三元组通常表示为 $\{Pre-condition\} \ m() \ \{Post-condition\}$ ，其中 $m()$ 表示具体的系统操作。通过这种形式，可以清晰地描述系统操作执行前后的状态约束。前置条件用于说明系统操作执行之前必须满足的状态假设。就系统运行来说，这些条件可能包括某些对象在操作开始之前必须已经存在，某些对象的属性值需要满足特定要求，并且某些对象之间需要已经建立特定连接关系。只有当这些条件成立时，系统操作才能被正确执行。后置条件则用于描述系统操作完成之后系统状态所发生的变化。操作执行结束后，系统可能会产生新的对象，也可能删除已有对象；部分对象的属性值可能被更新；对象之间的连接关系也可能被建立或者解除。通过明确这些变化，可以完整地表达系统操作对系统状态产生的影响。在实际应用中，系统操作契约不仅有助于明确操作的输入输出与状态变化，还能为自动化验证、测试设计以及需求与设计一致性检查提供依据。通过对每个操作的前置条件与后置条件进行详细描述，我们可以在早期阶段发现潜在逻辑冲突或遗漏，从而降低开发过程中的错误风险。我们基于霍尔三元组可以定义系统操作的契约，如表 2.7 所示本文采用如下模版：

表 2.10 “预约挂号”用例的“提交支付订单”系统操作契约
Table2. 10 The operation contract of "submitRegistration" for "Appointment Registration" use case

系统操作契约	内容
系统操作	当前契约所属的系统操作
交叉引用	当前系统操作所属的用例
前置条件	系统操作在执行之前必须满足的假设条件
后置条件	系统操作引发系统发生状态变化后所呈现的结果

系统操作契约定义的实例如表 2.8 所示，这里我们展示了“预约挂号用例的

“提交支付订单”系统操作契约。

表 2. 11 “预约挂号”用例的“提交支付订单”系统操作契约

Table2. 11 The operation contract of "submitRegistration" for "Appointment Registration" use case

case	
系统操作契约	内容
系统操作	<code>submitRegistration(p_id: string, s_id: string)</code>
交叉引用	预约挂号（用例）
前置条件	■ <code>Patient</code> 对象（当前操作者）必须存在且已验证。 ■ <code>TicketSlot</code> 对象必须存在，且其 <code>remaining_count</code> 属性必须大于 0 ■ 创建了一个新的 <code>RegistrationOrder</code> 对象。 ■ <code>RegistrationOrder</code> 对象的 <code>status</code> 属性初始化为“待支付”
后置条件	■ <code>RegistrationOrder</code> 对象与当前的 <code>Patient</code> 对象之间创建了连接关系 ■ <code>RegistrationOrder</code> 对象与对应的 <code>TicketSlot</code> 对象之间创建了连接关系 ■ <code>TicketSlot</code> 对象的 <code>remaining_count</code> 属性减 1